

Mandate

Agent-to-agent B2B purchasing on Razorpay, where every money action is bounded, gated and provable.

BUILT BY

Dev Chalana · AI Growth & Agentic Commerce track

REPOSITORY

github.com/dev591/razorpay

TRY THIS IN TEN SECONDS

Press “Tamper”

A locked cart is mutated and re-verified in front of you. It returns the expected hash, the tampered hash, and the rejection. Settlement refuses.

Press “Break the model”

The language model becomes genuinely unreachable. Five vendors degrade, a winner is still selected, and a real Razorpay order is created at 14.99% margin against a 9% floor.

Those two buttons are the whole submission in miniature: the record cannot be edited, and the money rules hold with no model in the loop. Everything after this page explains how.

01 What I set out to prove

An agent that can spend money is only useful if it can also be stopped, bounded, and audited. It is not hard to get a language model to produce a plausible purchase order. What is hard is being able to say, afterwards, exactly what was agreed, why, under what limits — and prove that none of it changed since.

So I worked to one rule: the model proposes, the protocol disposes. Every constraint that protects money is arithmetic in code, never an instruction in a prompt. A model that is adversarial, hallucinating, or completely offline cannot breach any of them.

Three things follow from that, and they shape everything below.

- 1 **Determinism beats persuasion.** Margin floors, budget ceilings, delivery windows and basket contents are checked in code before any model output is allowed to influence anything.
- 2 **The record is the product.** The audit trail is a hash-chained append-only ledger, so it cannot be quietly edited — including by me.
- 3 **Degraded is a state, not a failure.** The system completes real orders with the language model entirely unreachable, and labels every cart it produced that way.

02 Growing the merchant's revenue

Enterprises will never delegate real spend to autonomous agents that can hallucinate or breach margin. Security is not a constraint on growth here — it is the precondition for it: a merchant only gets access to agentic demand once it is provably safe to sell to. So here is the money side stated as money, using figures the system meters rather than estimates.

Turning payment terms into a paid growth lever

The floor is enforced on net realisable revenue, not sticker price, so extending net-15/30/45 to cash-strapped SMB buyers is priced automatically rather than quietly eaten. That unlocks sales from buyers who couldn't clear an advance-only floor — without the merchant ever conceding real margin to get there. In the fully degraded run the winning cart landed at **14.99% against the merchant's own 9% floor** — six points of headroom retained with no model in the loop at all. Freight is a separate pass-through line, so reported margin is margin on goods rather than a rush charge in disguise.

An automated AOV lift on every autonomous settlement

The upsell engine hooks into leftover buyer intent on every negotiation: it only attaches a line when the base cart already clears 20 points of margin headroom, the extra fits inside the buyer's remaining budget, and the SKU is not already in the cart. That means the AOV lift is risk-free by construction — it cannot push a deal under the floor, and it cannot break the budget that got the intent approved. Because the rule is arithmetic rather than a prompt, the same cart produces the same upsell every time, so a merchant can audit exactly why it was offered.

Market scale that doesn't explode acquisition cost

The shortlist gate is what lets the system evaluate a large supplier base and connect a buyer to the right transaction without runaway API spend as the marketplace grows: cost per purchase stays flat in the number of merchants, not linear in it. Every completion's usage block is recorded, so this isn't a claim — it's measured: roughly **₹0.6–0.8 per multi-vendor negotiation**, around nine model calls per vendor. One three-of-seven run reported **36 model calls and ₹0.63 saved, measured**, against a B2B order size where that is rounding error. Agent-mediated selling only scales for a merchant if the cost of quoting is knowable, and here it is on the dashboard.

Growth for the marketplace, not just the cheapest three merchants

A purely greedy shortlist means the same handful of merchants win every auction and nobody else ever transacts. For a platform whose purpose is growing *merchants'* revenue, that is an anti-feature, and it is a growth problem before it is a fairness one. An exploration slot reserved for the vendor with the fewest orders is implemented and unit-tested; it defaults to zero purely so a reviewer comparing the bounds to the selection sees them agree exactly. Turning it on is a one-constant change.

Revenue reporting a merchant can trust

Booked GMV — signed mandates with real orders behind them — is tracked separately from money actually captured, and the leaderboard is maintained incrementally so a settlement updates the ranking correctly rather than approximately. A dashboard that presents booked GMV as revenue is how merchants get misled about their own business.

Then the other half of the brief, made literal: a merchant on this system is transactable by an AI buyer with no human and no UI. There is an agent card at a well-known URL declaring capabilities and guarantees, cross-vendor term search and trie-backed prefix completion so a buyer does not have to pull every catalog and filter client-side, the gated intent lifecycle, a per-session event stream, and the audit chain — all as ordinary HTTP.

03 How it is put together

Five layers, and dependencies only ever point downward. The bottom layer is the one that has to stay trustworthy no matter what an agent does, so nothing above it can reach in.

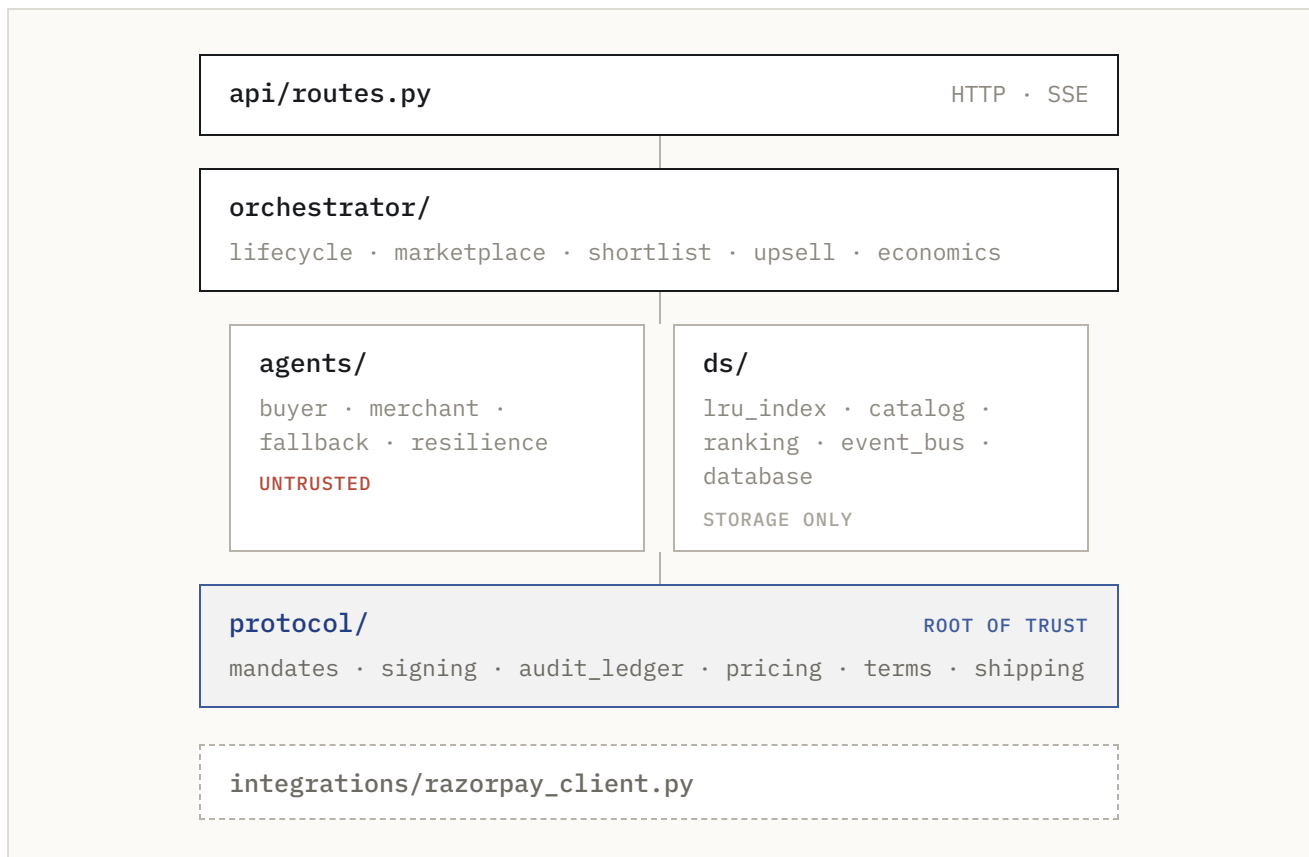


Fig. 1 — The model sits inside a box whose walls are all deterministic. It can choose how to negotiate; it cannot choose whether a bound applies.

Where a protocol-level check genuinely needs catalog data — budget feasibility, basket availability — the import happens lazily inside the function, specifically so the dependency direction is not inverted at module scope. Everything an agent produces is re-validated

before it is used, and the frontend is treated as display only: every constraint is checked again on the server.

04 The mandate chain

Modelled on the AP2 idea of a chain of signed mandates, where each stage commits to the one before it.

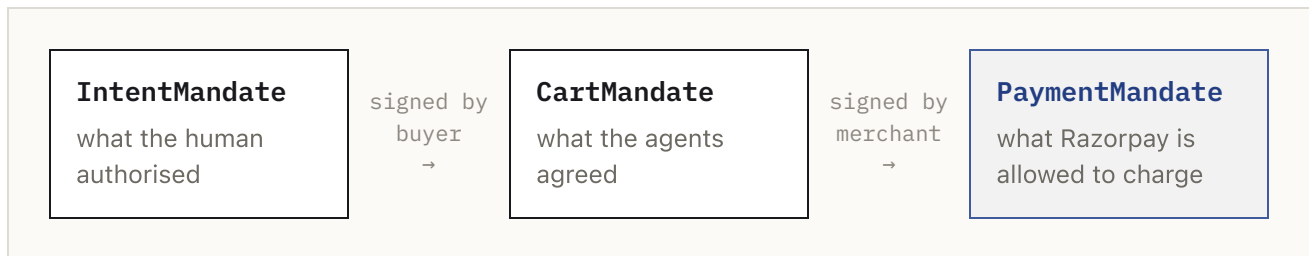


Fig. 2 – Each mandate is HMAC-signed over canonical JSON, so the same logical object always hashes identically.

The intent signature covers the basket, not just the money

The signature spans every bound the agent must not exceed: budget, quantity band, delivery deadline, credit ceiling, and the named lines themselves. Signing only price and quantity would leave terms and goods unprotected, and an approved mandate could then be settled on different credit terms — or a completely different set of goods that the human never saw.

```
intent.signature = sign("buyer_agent", {
  "id", "max_spend", "qty_min", "qty_max",
  "ship_within_days", "preferred_payment_terms",
  "requested_lines": [{"name", "qty"}, ...],
})
```

The hash lock before payment

The agreed cart is content-hashed with SHA-256 over canonical JSON. The payload includes every line item, the upsell line, and a synthetic line carrying freight and ETA — locking only the line items would let a settled order be quietly re-shipped on a slower, cheaper service without the lock noticing. `POST /sessions/{id}/tamper` mutates a locked cart and re-verifies, returning the expected hash, the tampered hash, and the rejection.

The ledger

Append-only JSONL, one file per session, hash-chained. Each entry's hash covers its own payload plus its predecessor's hash, so mutating or deleting any past entry breaks every hash after it. Tampering becomes mechanically detectable rather than a matter of policy.

```
entry = {seq, timestamp, event_type, payload, prev_hash}
entry["hash"] = sha256(canonical_json(entry))
```

I kept it out of SQLite on purpose: a hash chain *is* the shape of an append-only file, and the chain — not the storage engine — is what makes it tamper-evident. The one real trap is concurrency. Several vendors negotiate in parallel threads against the same session ledger, so read-last-hash-then-append has to be atomic. There is an internal lock for that, but callers must also reuse a single ledger instance per session across threads; a second instance would read a stale in-memory tail and corrupt the chain regardless of the lock. That is documented at the class, because it is the kind of thing the next person will get wrong.

Verification does not trust the server. The README carries a one-liner that recomputes every hash from genesis rather than reading the server's own `chain_valid` flag.

05 The commercial model

This is the part most demos skip, and it is where the interesting bugs lived.

There were two definitions of margin

Pricing clamped to a markup on cost. Reporting measured margin on revenue. Both were individually correct; comparing them was not. A 12% floor clamped a ₹100-cost item to ₹112, which reports as $12/112 = 10.71\%$ — under its own floor, for every item, on every round.

BEFORE

14 spurious floor violations per session.
Only 1 of 3 vendors ever converged.

AFTER

0 violations. 3 of 3 vendors converging.

The constant is named *margin*, so margin-on-revenue won, and the pricing side was inverted to match: `price = cost / (1 - margin)`. The rule I now hold to is that no margin formula is ever inlined — everything goes through `min_sellable_price()` and `margin_pct()` in one module, and the near-floor buffer has exactly one home shared by the orchestrator and the offline fallback. A second copy is precisely how the two definitions drifted apart in the first place.

Payment terms are a priced lever

A rupee collected in 45 days is not a rupee. A seller quoting the same sticker price on advance and on net-45 is quietly earning several points less margin, and a floor checked against sticker price would never notice. So financing cost is modelled explicitly, and the floor is enforced on net realisable revenue — what the seller actually banks.

CONSTANT	VALUE	WHY THAT NUMBER
WORKING_CAPITAL_APR	18.0%	Indian SMB working-capital lines run in the high teens
MSMED_MAX_CREDIT_DAYS	45	A legal ceiling, not a tuning knob — MSMED Act, s.43B(h)
ADVANCE_DISCOUNT_PCT	2.0%	The classic "2/10" cash discount

```
financing_cost(amount, terms) = amount × (APR/100) × (days/365)
min_price = cost / ((1 - floor/100) × (1 - APR/100 × days/365))
```

Without this, a seller could hold the floor on paper while conceding real margin through the terms instead.

Freight is its own line, and does not earn margin

Originally the cost of fast delivery was folded into unit price as an expedite premium. No real quote works that way, and burying it corrupted the margin story, because a rush charge is not gross margin on goods. Freight is now a pass-through: added to the invoice, excluded from the margin the floor is enforced against. A merchant does not earn margin on someone else's diesel.

The freight number itself is a function of both the vendor's actual position and the requested ETA together, not ETA alone — the same delivery window costs differently depending on how far the vendor actually is from the buyer. That is what makes the multiplier curve below meaningful rather than a flat rush-fee table.

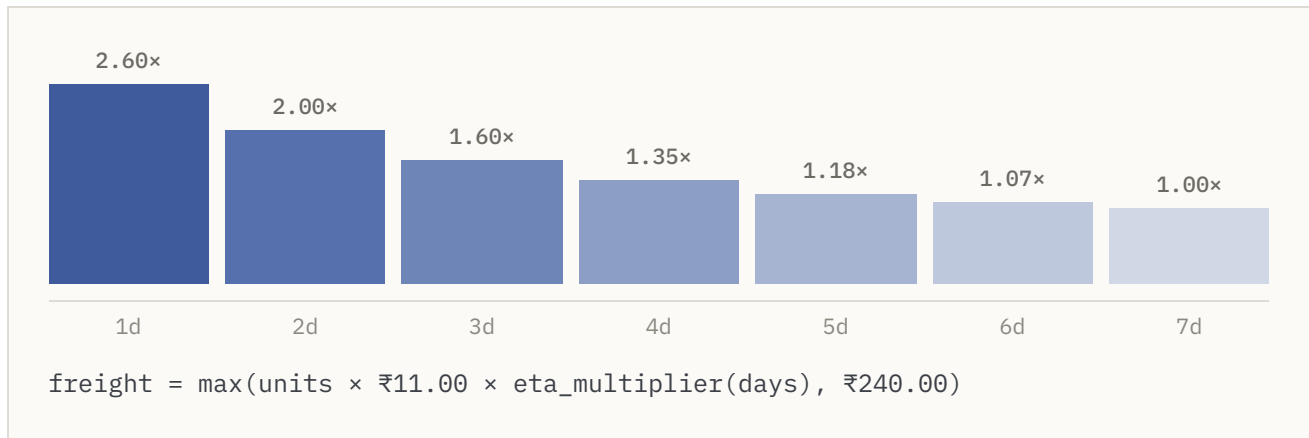


Fig. 3 — ETA multipliers. Steep at the short end because next-day is a dedicated vehicle, not a slot on a shared one.

The upsell engine is rule-based on purpose

No model decides the upsell, because the decision has to be explainable and reproducible. It fires only when the base cart's margin already clears 20 points of headroom, the extra still fits inside the buyer's remaining budget, and the SKU is not already in the cart.

06 Negotiation, and what the model is not allowed to decide

Six rounds per vendor, all vendors running concurrently against one shared ledger. Each round: the merchant proposes, the buyer evaluates, then accepts, counters or walks. Before any of that reaches the model, three checks run in plain code.

CHECK	WHAT IT PREVENTS
<code>total_price > max_spend</code>	Overspend
<code>lead_time > ship_within</code>	A cart arriving after it is useful is a different deal, not a cheaper one
<code>basket_shortfall()</code>	A vendor substituting whatever it is cheapest at for what was actually asked for

The model decides how to push back — wording and strategy — never whether a bound was breached. Two details in the basket check matter: it matches on item name, case-folded, never SKU, because each vendor sells the same goods under its own code and SKU equality would fail for every vendor but one; and the upsell line is deliberately excluded from satisfying the basket, since an extra the merchant attached cannot count toward what the buyer asked for.

Merchant proposals use JSON-schema structured output with the vendor's SKU list injected as an enum, so the model cannot name a product the vendor does not stock. The prompt is explicit that the `items` array is what actually gets charged, not the prose around it, because models get arithmetic in sentences wrong. Requests are bounded at 30 seconds — load-bearing, because the marketplace blocks on all vendors finishing, so without it one stalled call would hang the whole request for the SDK's ten-minute default.

07 The shortlist gate

Negotiating with a vendor costs around nine model calls, so broadcasting to every merchant makes one purchase linear in the size of the marketplace. At a hundred thousand merchants that is roughly nine hundred thousand model calls for a single restock. This is a spend problem before it is a latency problem, which is why the gate runs before the fan-out rather than ranking the survivors of an expense already incurred.

Two properties make it defensible rather than a guess.

1 — THE KEY IS A PROVABLE LOWER BOUND

Vendors are ranked by the cheapest total they could quote without breaching their own margin floor. No amount of negotiation moves that number, so if an excluded vendor's bound exceeds the winning price, that vendor could not have won.

List price is the wrong key: it ranks who *starts* cheap, not who *ends* cheap. QuickSupply lists at ₹880 and Meridian at ₹885 — sticker price favours QuickSupply. But on cost and floor, QuickSupply bottoms out at ₹717.39 while Meridian reaches ₹648.35.

After the winner is known, `verify_admissible()` re-checks the claim and writes the result to the ledger every time, including when it fails — a shortlist that skipped a cheaper vendor is exactly the fact an audit trail owes the buyer.

2 — IT REPORTS WHAT IT SAVED

Per-vendor cost is derived from metered usage, not estimated, so the console can say *"shortlisted 3 of 7 · saved ~48 model calls · ₹0.86 (measured)"*.

There is a real tension I want to name rather than hide. A purely greedy shortlist means the same three merchants win every auction and nobody else ever transacts, which for a platform whose purpose is growing merchant revenue is an anti-feature. An exploration slot reserved for the vendor with the fewest orders is implemented and unit-tested, but defaults to zero, so the shortlist is exactly "the k cheapest achievable" and a reader comparing bounds to the selection sees them agree. It is one constant away from being switched on.

Scoring, and who actually decides

Converged offers are scored on price, delivery and terms, weighted by the buyer's own stated priorities (0.5 / 0.3 / 0.2 by default). How fast a vendor caved is worth a 3% tiebreaker and nothing more — it was originally weighted as half of "speed", which let the most expensive vendor win purely for conceding quickest. How quickly a vendor caved is a property of the negotiation, not of the deal the buyer ends up with; the buyer's speed preference is about delivery.

An offer below half the median of converged offers is flagged and discounted in proportion to how extreme it is, so a gamed near-zero price cannot win on raw cheapness. And the scorer produces a recommendation, not a verdict: every converged cart is retained and can be settled against the same signed intent, with the human's override written to the ledger alongside the recommendation it replaced.

08

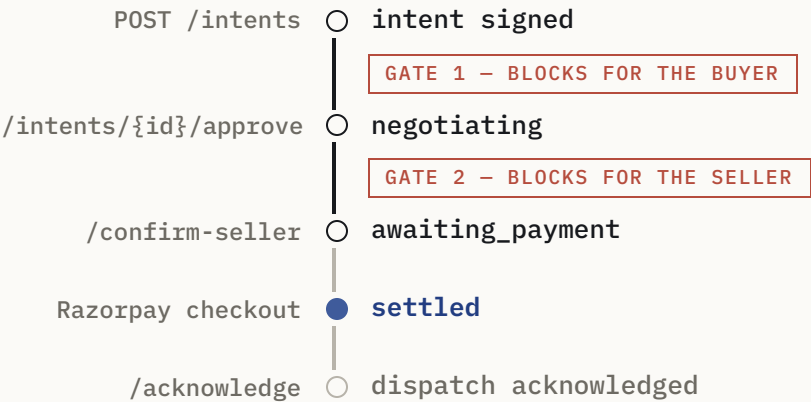


Fig. 4 – There is no code path from an unapproved intent to a payment.

Gate 1. Creating an intent mints and signs it, then halts. Nothing reaches a model, a vendor or Razorpay until a human approval is recorded against the intent hash.

Gate 2. Every converged cart parks for the merchant to accept — and the merchant can decline it outright, in which case the buyer's other converged offers stay settleable, so a runner-up carries the deal instead. A gate that can only say yes is not a gate. This used to be conditional on the margin sitting near the floor, which optimises the wrong risk: the agent negotiated against a catalog, and a catalog is a claim about stock rather than a fact. Real merchants oversell and under-update. Nobody is harmed by a deal waiting for a human yes; a buyer paying for forty units a vendor cannot ship is a real failure. The near-floor test still runs, but as the *reason* shown to the merchant — “confirm you can ship this” reads differently from “this one is thin, look closely”.

THE BUG BEHIND GATE 2

Seller confirmation originally read only an in-memory dict, so every deal parked before a process restart was permanently unconfirmable — 274 sessions were stuck. It now rebuilds the cart mandate from the session persisted on disk. Checked against real rows, the rebuilt cart reproduces an identical hash and signature, because both are functions of field values rather than object identity. The old code comment asserting a cart “cannot round-trip” through JSON was simply wrong.

08b Cost and system design at scale

Pulling the economics into one place: what one purchase costs, why it does not grow linearly with the marketplace, and where the design breaks first.

LEVER	EFFECT, MEASURED
Shortlist gate before fan-out	Without it, cost per purchase is linear in marketplace size — ~9 model calls per vendor, so 100k merchants is ~900k calls for one restock. With it, cost is bounded by k regardless of how many merchants exist. A measured 3-of-7 run: ~48 model calls, ~₹0.86 saved.
30-second per-call timeout	The marketplace blocks on every vendor finishing, so one stalled call otherwise hangs the whole request for the SDK's 10-minute default. Bounds worst-case negotiation latency to a known ceiling.
Reject infeasible requests pre-model	A budget/basket check that fails costs one comparison, not six rounds of negotiation ending in no offer. Named-shortfall rejection is both cheaper and a better answer.
Catalog lookups (trie + inverted index)	$O(\text{len}(\text{prefix}) + \text{matches})$ instead of scanning every listing — the cost that scales with catalog size stays off the model-call budget entirely.
LRU session cache over SQLite	$O(1)$ get/put/evict keeps hot sessions off disk without an unbounded process-memory footprint; SQLite stays the source of truth so a miss costs one disk read, not lost data.

Where this breaks: sorting and indexing stay microseconds at any size that matters here — the wall is always model calls, which is why the shortlist gate is the one number I'd defend under scaling pressure, not the heap or the trie. Beyond a few thousand merchants the registry moves off in-process structures into the database and admissible-price bounds get pre-computed into the catalog index at write time rather than recomputed per request; beyond that, the single-process assumption — one event bus, one ledger writer per session — is what breaks first, and the fix is sharding by session id, not a rewrite.

09 Data structures, and why each one

Five structures, each picked for a specific property rather than for variety.

STRUCTURE	COST	WHAT IT REPLACED, AND WHY IT MATTERS
SessionStore LRU over disk-backed corpus	$O(1)$ get/put/evict $O(k)$ by buyer or seller	An unbounded dict plus a scan-and-filter per query. Eviction drops only the hot copy; SQLite stays authoritative, so a miss falls through to disk rather than losing data.
CatalogIndex trie + inverted index	$O(\text{len}(\text{prefix}) + \text{matches})$ $O(\text{rarest posting list})$	This is what makes the catalog agent-readable at scale. An AI buyer should not have to pull every business and grep client-side. Prefix search cost is independent of catalog size.
top_k_offers bounded heap	$O(n \log k)$	Sorting everything to read the top few. One pass, k-sized heap.
Leaderboard sorted list, bisect insert	$O(\log n)$ write $O(1)$ read top-k	A heap cannot maintain this — a settlement changes a vendor's cumulative revenue and there is no efficient decrease-key. Read on every dashboard poll, written once per settlement, so the trade is the right way round.
EventBus ring buffer + fan-out	$O(\text{subscribers})$, never blocks	Creating a session used to block for 10–20 seconds and return everything at once, throwing away every interesting event inside that window. Bounded dequeues cannot leak; a slow consumer applies backpressure only to itself.

Two small things I am glad I got right: vendor cost is the merchant's own input and never leaves the building — agents only ever see list price. And booked GMV, meaning signed mandates with real orders behind them, is tracked separately from money actually captured, because a dashboard must never present the first as revenue.

10 When things break

Retries separate transient from terminal. A dropped socket, a 429 or a 5xx is worth retrying; a malformed request or an auth failure is not, and propagates immediately rather than burning the clock. Backoff is bounded and fully jittered, which matters specifically because every vendor thread fails at the same instant on a network blip — without jitter they would retry in lockstep and hammer the same recovering endpoint.

When retries are exhausted, there is no model in the loop at all. The offline path fills the quantity band or the named basket from the vendor's catalog, prices at a round-appropriate target margin, walks price toward the floor if it is over budget, and only then trims quantity — never below a quantity the buyer explicitly named, and never below the floor. An offer that breaches the vendor's own limit is worse than no offer. Every degraded cart is flagged in the cart, in the UI and in the audit trail with a stated reason; healthy carts carry `degraded: false` explicitly, so “not degraded” is a positive assertion rather than an absence.

BREAKING IT ON PURPOSE

There is a chaos switch — and a *Break the model* button in the console — that makes the model genuinely unreachable. It breaks the upstream, not the outcome: the same connection error the SDK raises is raised, so the failure classification, the rule-based quoting and the degraded labelling all execute exactly as they would in a real outage. It skips the retry ladder on purpose — retrying a failure we injected ourselves recovers nothing; a genuine SDK failure still gets all three attempts. A switch that jumped straight to the fallback would demonstrate nothing. It self-expires after five minutes, because a demo control left on by accident eventually makes a working system look broken.

Result with the model fully dead: three shortlisted vendors degraded at round one, a winner selected, a real Razorpay order created, margin at 14.99% against the winning vendor's 9% floor, chain valid, every cart flagged.

Failure-mode matrix

FAILURE	RESPONSE	RESULTING STATE
Model unreachable	Rule-based quote after three tries	degraded, order completes
Model malformed or auth	Propagate immediately	vendor drops, others continue
Razorpay order or capture fails	Mandate held intact, 30s bound	payment_provider_error
Bad checkout signature	Refuse capture	payment_signature_invalid
Tampered cart	Hash mismatch, refuse settlement	rejected_hash_mismatch
Budget or basket infeasible	Reject before any model call, with the arithmetic	400 + explanation
Process restart mid-deal	Rehydrate cart from persisted session	confirmable
SSE stream dies	Frontend watchdog polls session state	recovers across a restart

Rejecting up front with the actual shortfall is both cheaper and a far better answer than six rounds of model calls ending in “no vendor responded”. So the budget check names the number: *“a budget of ₹42,000 cannot buy 20× wireless mouse and 20× mechanical keyboard — the cheapest vendor can only reach ₹55,652.20 at its own margin floor.”*

11 Payments

Razorpay test mode: create order, hosted checkout, callback, HMAC signature verification, capture. Every call is wrapped in its own retry with a 30-second bound, so a slow provider becomes a clean status rather than a hung request.

The bug worth admitting here is the confirmation page. It printed “payment captured” from a bare `.then()` — including when signature verification had failed — because the callback returns HTTP 200 for every outcome and puts the real result in the session status. A buyer whose payment was rejected would have been told it succeeded. It now branches on status and renders distinct screens for settled, invalid signature and failed capture, plus a handler for Razorpay's own payment-failed event.

Route attribution is simulated and labelled as such: linked-account ids are derived deterministically so payout intent is explicit in the mandate and the audit trail rather than implicit, but settlement goes through the single real test account. A real linked account would be issued by Razorpay at KYC time.

12 The console

Next.js and React on the frontend, with five routes: a landing page that explains, a buyer console, a merchant console, a vendor registry with agent catalog search, and an engineering page carrying the complexity table and the measured economics.

The buyer console's state is folded from the event stream rather than polled. A negotiation is 10–20 seconds of concurrent model round-trips, and the whole point is showing it happen rather than freezing on a spinner and dumping a finished object at the end. Duplicate events are guarded by a seen-set, because the server replays its buffer on reconnect. The merchant console deliberately polls instead: a dashboard left open for an hour has to survive a backend restart, and an empty event bus after a restart is exactly what a stream cannot recover from.

Also worth recording, because it cost me an afternoon: number inputs could not be cleared. Binding a number straight to `value` turns every clearing keystroke into `Number("") === 0`, so the field re-rendered as "0" and the next digits read "035000".

13 Bugs found by testing, not by reading

I am including these because they are where the real difficulty was. Every one of them was found by running the thing, not by re-reading the code.

01 Two definitions of margin. 14 spurious violations a session and one of three vendors converging, until markup-on-cost and margin-on-revenue were reconciled.

02 Lead-time discount applied to the floor. It pushed the minimum below the floor itself: 16 violations and zero offers on any long-credit request.

- 03 **Cart totals never serialised.** A plain property is invisible to the model dump, so every persisted cart lacked its own total and the leaderboard was reading zero.
- 04 **Cart signed before the upsell was attached.** The signature covered a total that no longer matched the cart.
- 05 **Convergence weighted as half of "speed".** The most expensive vendor could win for caving fastest.
- 06 **Parked deals died on restart.** 274 sessions permanently unconfirmable, fixed by rehydrating from the persisted session.
- 07 **Checkout reported success on a failed signature.** HTTP 200 for every outcome, with the real result in the body.
- 08 **Non-atomic session writes.** A session could be silently lost on Ctrl-C; fixed by moving the store of record to SQLite.

14 What I verified

CHAIN Every hash recomputed from genesis independently, not trusting the server's own flag. Valid across every session tested.	FLOOR INVARIANT Zero margin-floor violations across sessions after the pricing fixes, down from fourteen per session.
DEGRADED END TO END Model fully dead; order still completes with a real Razorpay order, the floor still holds, every cart flagged.	SHORTLIST Unit tests for bound arithmetic, k enforcement, cheapest-present, admissibility in both directions, determinism per seed, empty catalog.
GATE UNCONDITIONALITY A synthetic 45% margin cart, far outside the near-floor buffer, confirmed to park anyway.	CLEAN CLONE Fresh virtualenv, install, boot, serve. Full UI pass across all five routes with zero console errors, zero tracebacks, zero 5xx.

15 Decisions I would defend

SQLite, not a hosted database	The submission is a repository link. A clean clone has to run with no external service and no credentials.
The ledger stays JSONL	A hash chain is already an append-only file. A database adds a dependency and no extra integrity.
Upsell is rule-based	It has to be explainable and reproducible, every time, to the same inputs.
Fallback is rules, not a cached response	Degraded still has to respect the vendor's floor.
Chaos breaks the upstream	Short-circuiting to the fallback would prove nothing about the real failure path.
Seller gate is unconditional	Catalogs are claims about stock, not facts about it.
Floor on net realisable revenue	Otherwise payment terms become a hidden margin leak.

16 Gaps, stated plainly

A reviewer will find these anyway, so I would rather point at them myself.

The revenue numbers come from a test-mode corpus. Every figure in section 02 is measured rather than estimated, but it is measured against synthetic vendors and my own traffic. Attach rate and incremental GMV only become real claims under live merchant traffic, and I would not present them as forecasts.

No public deployment. The console holds a 40–70 second event stream that serverless hosts buffer or time out, so a public URL needs a persistent container. It runs from a clean clone with no credentials at all — the negotiation completes on the rule-based path and every cart is labelled — and adding API keys shows the live model path on top of that.

Two of three in-memory caches are still restart-fragile. Parked deals are fixed; approving an intent or selecting a runner-up across a restart still fails, and the same rehydration approach would work.

No auth, rate limiting, multi-tenancy or idempotency keys, and a single master signing secret where production would issue a key per agent at registration. All of it is labelled as such in the code rather than left to be discovered.

On scale, the honest ordering is that sorting is microseconds at any size that matters here. The wall is always the model calls — which is why the shortlist gate, not the heap, is the line I care about, and why I framed it as a spend bound rather than a speed optimisation. Beyond a few thousand merchants the registry moves to the database and bounds get pre-computed into the catalog index; beyond that, the single-process assumption is what breaks first.

17 Against the brief

“Every money action explainable, bounded and gated. Show the audit trail and one failure handled gracefully.”

REQUIREMENT	WHERE IT LIVES
Transactable by an AI buyer, end to end	Intent → shortlist → negotiate → seller accept → Razorpay order → payment → dispatch
Agent-readable catalog	Agent card at a well-known URL, trie and inverted index, cross-vendor search and prefix completion
Upsell and cross-sell	A rule-based engine gated on margin headroom, remaining budget and cart contents
Gated	Two blocking human gates, with no path from an unapproved intent to a payment
Bounded	Margin floor, basket, budget and the MSMED credit cap — all arithmetic, none of it prompted
Explainable, with the trail shown	Hash-chained ledger, live trail in the console, independently verifiable from the README
One failure handled gracefully	Chaos switch → a real outage → the rule-based path → the floor still holds → a real order

If there is one idea I would want carried out of this document, it is the first one: the model proposes, and the protocol disposes. Everything else is a consequence.